

Chapter 18. Memory Efficiency

As [Chapter 7](#) described, the CLR is able to perform automatic memory management thanks to its garbage collector (GC). This comes at a price: when a CPU spends time on garbage collection, that stops it from getting on with more productive work. On laptops and phones, GC work drains power from the battery. In a cloud computing environment where you may be paying for CPU time based on consumption, extra work for the CPU corresponds directly to increased costs. More subtly, on a computer with many cores, spending too much time in the GC can significantly reduce throughput, because many of the cores may end up blocked, waiting for the GC to complete before they can proceed.

In many cases, these effects will be small enough not to cause visible problems. However, when certain kinds of programs experience heavy load, GC costs can come to dominate the overall execution time. In particular, if you write code that performs relatively simple but highly repetitive processing, GC overhead can have a substantial impact on throughput.

To give you an example of the kinds of improvements that can sometimes be possible, early versions of Microsoft's ASP.NET Core web server framework frequently ran into hard limits due to GC overhead. To enable .NET applications to break through these barriers, C# introduced various features that can enable dramatic reductions in the number of allocations. Fewer allocations means fewer blocks of memory for the GC to recover, so this translates directly to lower GC overhead. When ASP.NET Core first started making extensive use of these features, performance improved across the board, but for the simplest performance benchmark, known as *plaintext* (part of the TechEmpower suite of web performance tests), this release improved the request handling rate by over 25%.

In some specialized scenarios, the differences can be even greater. I worked on a project that processed diagnostic information from a broadband provider's networking equipment (in the form of RADIUS packets). Adopting the techniques described in this chapter boosted the rate at which a single CPU core in our system could process the messages from around 300,000/s to about 7 million/s.

There is a price to pay, of course: these GC-efficient techniques can add significant complication to your code. And the payoff won't always be so large—although the first ASP.NET Core release to be able to use these features improved over the previous version on all benchmarks, only the simplest saw a 25% boost, and most improved more modestly. The practical improvement will really depend on the nature of your workload, and for some applications you might find that applying these techniques delivers no measurable improvement. So before you even consider using them, you should use performance monitoring tools to find out how much time your code spends in the GC. If it's only a few percent, then you might not be able to realize order-of-magnitude improvements. But if testing suggests that there's room for significant improvement, the next step is to ask whether the techniques in this chapter are likely to help. So let's start by exploring exactly how these techniques can help you reduce GC overhead.

(Don't) Copy That

The way to reduce GC overhead is to allocate less memory on the heap. And the most important technique for minimizing allocations is to avoid making copies of data. For example, consider the URL `http://example.com/books/1323?edition=6&format=pdf`. There are several elements of interest in here, such as the protocol (`http`), the hostname (`example.com`), or the query string. The latter has its own structure: it is a sequence of name/value pairs. The obvious way to work with a URL in .NET is to use the `System.Uri` type, as [Example 18-1](#) shows.

Example 18-1. Deconstructing a URL

```
var uri = new Uri("http://example.com/books/1323?edition=6&format=pdf");
Console.WriteLine(uri.Scheme);
```

```
Console.WriteLine(uri.Host);  
Console.WriteLine(uri.AbsolutePath);  
Console.WriteLine(uri.Query);
```

It produces the following output:

```
http  
example.com  
/books/1323  
?edition=6&format=pdf
```

This is convenient, but by getting the values of these four properties, we have forced the `Uri` to provide four `string` objects in addition to the original one. You could imagine a smart implementation of `Uri` that recognized certain standard values for `Scheme`, such as `http`, and that always returned the same string instance for these instead of allocating new ones, but for all the other parts, it's likely to have to allocate new strings on the heap.

There is another way. Instead of creating new `string` objects for each section, we could take advantage of the fact that all of the information we want was already in the string containing the whole URL. There's no need to copy each section into a new string, when instead we can just keep track of the position and lengths of the relevant sections within the string. Instead of creating a string for each section, we would need just two numbers. And since we can represent numbers using value types (e.g., `int` or, for very long strings, `long`), we don't need any additional objects on the heap beyond the single string with the full URL. For example, the scheme (`http`) is at position 0 and has length 4. [Figure 18-1](#) shows each of the elements by their offset and position within the string.

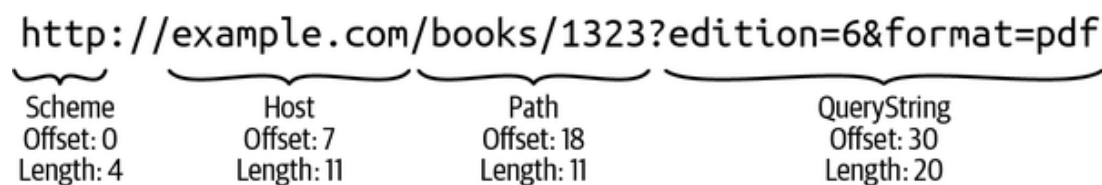


Figure 18-1. URL substrings

This works, but already we can see the first problem with working this way: it is somewhat awkward. Instead of representing, say, the `Host` with

a convenient `string` object, which is easily understood and readily inspected in the debugger, we now have a pair of numbers, and as developers, we now have to remember which string they point into. It's not rocket science, but it makes it slightly harder to understand our code, and easier to introduce bugs. But there's a payoff: instead of five strings (the original URL and the four properties), we just have one. And if you're trying to process millions of events each second, that could easily be worth the effort.

Obviously this technique would work for a more fine-grained structure too. The offset and position `(25, 4)` locates the text `1323` in this URL. We might want to parse that as an `int`. But at this point we run into the second problem with this style of working: it is not widely supported in .NET libraries. The usual way to parse text into an `int` is to use the `int` type's static `Parse` or `TryParse` methods. Unfortunately, these do not provide overloads that accept a position or offset within a `string`. They require a string containing only the number to be parsed. This means you end up writing code such as [Example 18-2](#).

Example 18-2. Defeating the point of the exercise by using `Substring`

```
string uriString = "http://example.com/books/1323?edition=6&format=pdf";  
int id = int.Parse(uriString.Substring(25, 4));
```

This works, but by using `Substring` to go from our (offset, length) representation back to the plain `string` that `int.Parse` wants, we've allocated a new `string`. The whole point of this exercise was to reduce allocations, so this doesn't seem like progress. One solution might be for Microsoft to go through the entire .NET API surface area, adding overloads that accept offset and length parameters in any situation where we might want to work with something in the middle of something else (either a substring, as in this example, or perhaps a subrange of an array). In fact, there are examples of this already: the `Stream` API for working with byte streams has various methods that accept a `byte[]` array, and also offset and length arguments to indicate exactly which part of the array you want to work with.

However, there's one more problem with this technique: it is inflexible about the type of container that the data lives in. Microsoft could add an overload to `int.Parse` that takes a `string`, an offset, and a length, but it would only be able to parse data inside a `string`. What if the data happens to be in a `char[]`? In that case, you'd have to convert it to a `string` first, at which point we're back to additional allocations. Alternatively, every API that wants to support this approach would need multiple overloads to support all the containers that anyone might want to use, each potentially requiring a different implementation of the same basic method.

More subtly, what if the data you have is currently in memory that's not on the CLR's heap? This is a particularly important question when it comes to the performance of servers that accept requests over the network (e.g., a web server). Sometimes, data received by a network interface won't be delivered directly into memory on .NET's heap. Also, some forms of interprocess communication involve arranging for the OS to map a particular region of memory into two different processes' address spaces. The .NET heap is local to the process and cannot use such memory.

C# has always supported use of external memory through *unsafe code*, which supports raw unmanaged pointers that work in a similar way to pointers in the C and C++ languages. However, there are a couple of problems with these. First, they would add yet another entry to the list of overloads that everything would need to support in a world where we can parse data in place. Second, code using pointers cannot pass .NET's type safety verification rules. This means it becomes possible to make certain kinds of programming errors that are normally impossible in C#. It may also mean that the code will not be allowed to run in certain scenarios, since the loss of type safety would enable unsafe code to bypass certain security constraints.

To summarize, it has always been possible to reduce allocations and copying in .NET by working with offsets and lengths and either a reference to a containing string or array or an unmanaged pointer to memory, but there was considerable room for improvement on these fronts:

- Convenience

- Wide support across .NET APIs
- Unified, safe handling of the following:
 - Strings
 - Arrays
 - Unmanaged memory

.NET offers a type that addresses all of these points: `Span<T>`. (See the sidebar, [“Support Across Language and Runtime Versions,”](#) for more information on how the features described in this chapter relate to C# language and .NET runtime versions.)

SUPPORT ACROSS LANGUAGE AND RUNTIME VERSIONS

`Span<T>` is built into .NET and is available to any library that targets .NET Standard 2.1. You can also use it on .NET Framework via a NuGet package, `System.Memory`, but be aware that this package has some limitations.

First, although this NuGet package adds `Span<T>` and related types, it cannot modify existing libraries. To fulfill the “wide support across .NET APIs” requirement, Microsoft added numerous methods to the .NET runtime libraries. For example, new overloads of `int.TryParse` accept `ReadOnlySpan<char>` as an alternative to `string`. The `System.Memory` NuGet package can’t add new static methods to `int`, so these new methods are not available in .NET Framework.

Second, this package provides a slightly different implementation than the one you will get when running the exact same code on .NET. These newer runtimes enable a more efficient implementation of `Span<T>` and related types, and provide related optimizations. This is critical to the high performance offered by the features discussed in this chapter. The latest version of the .NET Framework at the time of writing (version 4.8.1) lacks the `Span<T>` optimizations, and Microsoft has no plans to add them in future versions because .NET supersedes the .NET Framework. Code using these techniques works correctly on .NET Framework, but if you want to reap the full performance benefits of these techniques, you’ll need to run on .NET.

Representing Sequential Elements with `Span<T>`

The `System.Span<T>` value type represents a sequence of elements of type `T` stored contiguously in memory. Those elements can live inside an array, a string, a managed block of memory allocated in a stack frame, or unmanaged memory. Starting with .NET 7.0 and C# 11.0, it is also possible to create a single-element span that refers to an individual field or variable without needing to use unsafe code. Let's look at how `Span<T>` addresses each of the requirements enumerated in the preceding section.

A `Span<T>` encapsulates both a pointer to the start of the data in memory and its length. To access the contents of a span, you use it much as you would an array, as [Example 18-3](#) shows. This makes it much more convenient to use than ad hoc techniques in which you define a couple of `int` variables and have to remember what they refer to.

Example 18-3. Iterating over a `Span<int>`

```
static int SumSpan(ReadOnlySpan<int> span)
{
    int sum = 0;
    for (int i = 0; i < span.Length; ++i)
    {
        sum += span[i];
    }
    return sum;
}
```

Since a `Span<T>` knows its own length, its indexer checks that the index is in range, just as the built-in array type does. The performance is very similar to using a built-in array. This includes the optimizations that detect certain loop patterns—for example, the CLR will recognize [Example 18-3](#) as a loop that iterates over the entire contents, enabling it to generate code that doesn't need to check that the index is in range each time around the loop. (On .NET Framework, `Span<T>` is a little slower than an array, because its CLR does not include the optimizations for `Span<T>`.)

You may have noticed that the method in [Example 18-3](#) takes a `ReadOnlySpan<T>`. This is a close relative of `Span<T>`, and there is an implicit conversion enabling you to pass any `Span<T>` to a method that takes a `ReadOnlySpan<T>`. The read-only form enables a method to declare clearly that it will only read from the span, and not write to it. (This is enforced by the fact that the read-only form's indexer offers just a `get` accessor, and no `set`.)

TIP

Whenever you write a method that works with a span and that does not mean to modify the span's data, you should use `ReadOnlySpan<T>`.

`Span<T>` defines an explicit conversion from arrays. Similarly, `ReadOnlySpan<T>` is implicitly convertible from arrays and also strings. This enables [Example 18-4](#) to pass an array to the `SumSpan` method. Of course, we've gone and allocated an array on the heap there, so this particular example defeats the main point of using spans, but if you already have an array on hand, this is a useful technique.

Example 18-4. Passing an `int[]` as a `ReadOnlySpan<int>`

```
int[] numberArray = [1, 2, 3];
Console.WriteLine(SumSpan(numberArray));
```

Although [Example 18-4](#) constructs an array, you may be surprised to discover that [Example 18-5](#) does not, despite appearing to construct an array explicitly.

Example 18-5. Array syntax implicitly creating a `ReadOnlySpan<int>` directly

```
Console.WriteLine(SumSpan(new int[] { 1, 2, 3 }));
```

When you use this array initialization syntax in a place where a `ReadOnlySpan<T>` is required, then as long as the initializer values are all constants, the compiler can perform an optimization: it does not create a

real .NET array. Instead, it embeds the initializer values directly into the compiled output as a block of binary data and generates code that obtains a `ReadOnlySpan<T>` that points directly to that data. This optimization relies on a couple of facts. First, spans provide no way to get hold of the underlying container. This means our code can't tell whether there really is an array behind the span, so it doesn't actually matter if no array gets created. Second, this relies on the data being read-only—since methods might run multiple times, it needs to be possible to create an identical span every time, and if we were using a writable span, it would not be possible to share the same underlying block of memory every time.

[Example 18-6](#) shows a couple of examples in which creating a span with the same array initializer syntax really will create an array. In the first case, not all of the values are constant, so the optimization just described can't be applied. And in the second case, we assign the array into a `Span<int>` (even though `SumSpan` needs only a `ReadOnlySpan<int>`) so the compiler generates code that creates an array to enable the data to be modified. If the only thing we're doing with this data is passing it to `SumSpan` (which is allowed because `Span<T>` is implicitly convertible to `ReadOnlySpan<T>`) that's a waste, because the array will never be modified in practice.

Example 18-6. Array syntax examples that defeat the no-array optimization

```
// Creates an array because one of the initializer values is not constant.
Console.WriteLine(SumSpan(new int[] { 1, 2, DateTime.Now.Hour }));

// Creates an array because we asked for a Span<int> (not read-only).
Span<int> numberSpan = new int[] { 1, 2, 3 };
Console.WriteLine(SumSpan(numberSpan));
```

It is possible to avoid creating a real array even in these scenarios because `Span<T>` also works with stack-allocated arrays, as [Example 18-7](#) shows.

Example 18-7. Passing a stack-allocated array as a `ReadOnlySpan<int>`

```
ReadOnlySpan<int> nonConstReadOnly = stackalloc int[] { 1, 2, DateTime.Now.Hour };
Console.WriteLine(SumSpan(nonConstReadOnly));

Span<int> constWriteable = stackalloc int[] { 1, 2, 3 };
Console.WriteLine(SumSpan(constWriteable));
```

C# disallows most uses of `stackalloc` outside of code marked as `unsafe`. This allocates memory on the current method's stack frame, so this array won't have the usual .NET object headers that an ordinary array on the GC heap would have—it's just the raw values. This means that a `stackalloc` expression produces a pointer type, `int*`, in this example. You can normally only use pointer types directly in unsafe code blocks. However, the compiler makes an exception to this rule if you assign the pointer produced by a `stackalloc` expression directly into a span. This is permitted because spans impose bounds checking, preventing undetected out-of-range access errors of the kind that normally make pointers unsafe. Also, `Span<T>` and `ReadOnlySpan<T>` are both defined as `ref struct` types, and as [“Stack Only”](#) describes, this means they cannot outlive their containing stack frame. This guarantees that the stack frame on which the stack-allocated memory lives will not vanish while there are still outstanding references to it. (.NET's type safety verification rules include special handling for `ref`-like types such as spans.)

As you saw in earlier chapters, C# 12.0 adds *collection expressions*, a new syntax for creating and initializing collections. These work with spans, as [Example 18-8](#) shows. The first two collection expressions compile into code that puts the relevant data on the stack. The final one passes a collection made entirely of constant values to a method taking a `ReadOnlySpan<int>`, so in this case the compiler uses the same trick we saw with array initializers: it just embeds the constant values as a block of binary data in the compiled output, and creates a span pointing directly to that.

Example 18-8. Using collection expressions with spans

```
// Lives on stack (because one of the values is not constant).
Console.WriteLine(SumSpan([1, 2, DateTime.Now.Hour]));
```

```
// Lives on stack (because using Span<int> means this must be writable).
Span<int> numbersCollectionExpression = [1, 2, DateTime.Now.Hour];
Console.WriteLine(SumSpan(numbersCollectionExpression));

// Pointer to constant data embedded in DLL.
Console.WriteLine(SumSpan([1, 2, 3]));
```

If a collection expression uses the spread syntax (`..`) to incorporate a copy of some other collection, the size of the resulting span is not fixed at compile time—it will be determined by however large that other collection is. In these cases the compiler falls back to generating code that puts the relevant collection on the heap to avoid risking a stack overflow. If you need a span that incorporates some data from other collections and you want to avoid heap allocation, you would use `stackalloc` directly, and not the collection expression syntax. If you do this, you should inspect the size of that incoming data because it's generally a bad idea to put more than a few hundred bytes of data on the stack. The default stack size is not the same on all platforms, and can be changed by configuration, but it's common for the stack to be 1.5 MB in size, and if you use `async` and `await` it can be hard to predict exactly how deep call stacks will get at runtime. It is wise to choose a fairly conservative upper size limit when using `stackalloc`, and you should either reject data that is too large, or fall back to a heap-based code path.

Earlier I mentioned that spans can refer to strings as well as arrays. However, we can't pass a `string` to our `SumSpan` method for the simple reason that it requires a span with an element type of `int`, whereas a `string` is a sequence of `char` values. `int` and `char` have different sizes—they take 4 and 2 bytes each, respectively. Although an implicit conversion exists between the two (meaning you can assign a `char` value into an `int` variable, giving you the Unicode value of the `char`), that does not make a `ReadOnlySpan<char>` implicitly compatible with a `ReadOnlySpan<int>`.¹ Remember, the entire point of spans is that they provide a view into a block of data without needing to copy or modify that data; since `int` and `char` have different sizes, converting a `char[]` to an `int[]` array would double its size. However, if we were to write a method accepting a `ReadOnlySpan<char>`, we would be able to pass it a `string`, a `char[]` array, or a `stackalloc char[]`, or could explicitly

construct a `ReadOnlySpan<char>` from an unmanaged pointer of type `char*` (because the in-memory representation of a particular span of characters within each of these is the same).

NOTE

Since strings are immutable in .NET, you cannot convert a `string` to a `Span<char>`. You can only convert it to a `ReadOnlySpan<char>`.

We've examined two of our requirements from the preceding section: `Span<T>` is easier to use than ad hoc storing of an offset and length, and it makes it possible to write a single method that can work with data in arrays, strings, the stack, or unmanaged memory. This leaves our final requirement: widespread support throughout .NET's runtime libraries. As [Example 18-9](#) shows, it is now supported in `int.Parse`, enabling us to fix the problem shown in [Example 18-2](#). The generic math feature added in .NET 7.0 defines this and a span-based `TryParse` in its `ISpanParsable<T>` interface, and since `INumberBase<T>` inherits from `ISpanParsable<T>`, span-based parsing is available for all numeric types.

Example 18-9. Parsing integers in a string using `Span<char>`

```
string uriString = "http://example.com/books/1323?edition=6&format=pdf";  
int id = int.Parse(uriString.AsSpan(25, 4));
```

`Span<T>` is a relatively new type (it was introduced in 2018; .NET has been around since 2002), so although the .NET runtime libraries now support it widely, many third-party libraries do not yet support it, and perhaps never will. However, it has become increasingly well supported since being introduced, and that situation will only improve.

Utility Methods

In addition to the array-like indexer and `Length` properties, `Span<T>` offers a few useful methods. The `Clear` and `Fill` methods provide convenient ways to initialize all the elements in a span either to the default value for the element type or a specific value. Obviously, these are not available on `ReadOnlySpan<T>`.

You may sometimes encounter situations in which you have a span and you need to pass its contents to a method that requires an array. Obviously there's no avoiding an allocation in this case, but if you need to do it, you can use the `ToArray` method.

Spans (both normal and read-only) also offer a `TryCopyTo` method, which takes as its argument a (non-read-only) span of the same element type. This allows you to copy data between spans. This method handles scenarios where the source and target spans refer to overlapping ranges within the same container. As the `Try` suggests, it's possible for this method to fail: if the target span is too small, this method returns `false`.

Collection Expressions and Spans

I've already shown how you can use C# 12.0's new collection expression feature to initialize a span, but there's a more subtle way that these two language features interact: spans can become involved when you initialize certain other collection types with collection expressions. Some collection types define a special *create method* that C# will use if you initialize those types with a collection expression. [Example 18-10](#) will use the `ImmutableList.Create` method, for example.

Example 18-10. Using a collection builder

```
using System.Collections.Immutable;

ImmutableList<int> numbers = [1, 2, 3, 4, 5];
```

Collection types can advertise a create method with the `[CollectionBuilder]` attribute. This method is required to accept a `ReadOnlySpan<T>` where `T` is the element type of the collection being initialized. As of .NET 8.0, only the immutable collection types define this. These types were previously quite awkward to initialize, and the simplest ways of creating them were not the highest-performing. But now, using a collection expression creates the most efficient possible initialization code.

Although not all collection types implement this, some types, such as `List<T>`, are known to the compiler, enabling it to generate tailored initialization code for those. But if you write your own collection type, and if its implementation details mean that you can perform initialization more efficiently if you can see all the elements up front instead of them being added one at a time, you can also implement a `create` method that accepts a `ReadOnlySpan<T>`, and declare that it is your creation method with the `[CollectionBuilder]` attribute.

Pattern Matching

Spans can be used with certain kinds of patterns. [Chapter 2](#) described list patterns, a new feature in C# 11.0, and as [Example 18-11](#) shows, you can use a span as the input to a list pattern.

Example 18-11. Using a span with a list pattern

```
static void CheckStart(ReadOnlySpan<char> chars)
{
    if (chars is ['H', .. ReadOnlySpan<char> theRest])
    {
        Console.WriteLine(theRest.Length);
    }
}
```

This pattern includes the `..` syntax, denoting a slice, which matches any elements in the source not explicitly specified in the pattern. This example has a declaration pattern to capture the elements matched by the slice into a new variable called `theRest`. As you can see, the captured slice is also a span. Spans are particularly well suited to list patterns that capture slices, because there's no need to make a copy of the relevant subsequence. If we had used `string` here instead of `ReadOnlySpan<char>`, the compiler would need to generate code that called `chars.Substring(1)` to obtain a suitable value for `theRest`, causing a new string to be allocated. This code avoids that, and yet we can still pass this `CheckStart` method a `string`, because `string` is implicitly convertible to `ReadOnlySpan<char>`.

Since C# 11.0, it has also been possible to use a `ReadOnlySpan<char>` (or a `Span<char>`) as the input to a string constant pattern, as [Example 18-12](#) shows. The compiler generates the code required to compare the span's contents with the string's contents.

Example 18-12. Span as input to a string constant pattern

```
static void RespondToGreeting(ReadOnlySpan<char> message)
{
    switch (message)
    {
        case "Hello":
            Console.WriteLine("Hello to you too");
            break;

        case "How do you do":
            Console.WriteLine("How do you do");
            break;
    }
}
```

Stack Only

The `Span<T>` and `ReadOnlySpan<T>` types are both declared as `ref struct`. This means that not only are they value types, they are value types that can live only on the stack. So you cannot have fields with span types in a `class`, or in any `struct` that is not also a `ref struct`. This also imposes some potentially more surprising restrictions. For example, it means you cannot use a span in a variable in an `async` method. These store all their variables as fields in a hidden type, enabling them to live on the heap, because asynchronous methods often need to outlive their original stack frame. In fact, these methods can even switch to a completely different stack altogether, because asynchronous methods can end up running on different threads as their execution progresses. For similar reasons, there are restrictions on using spans in anonymous functions and in iterator methods. You can use them in local methods, and you can even declare a `ref struct` variable in the outer method and use it from the nested one, but with one restriction: you must not create a delegate that refers to that local method, because this would cause the compiler to

move shared variables into an object that lives on the heap. (See [Chapter 9](#) for details.)

This restriction is necessary for .NET to be able to offer the combination of array-like performance, type safety, and the flexibility to work with multiple different containers. [“Representing Sequential Elements with Memory<T>”](#) will show what we can do instead in scenarios where this stack-only limitation is problematic.

Using ref with Fields

Before C# 11.0, the `Span<T>` and `ReadOnlySpan<T>` types were only able to exist thanks to special support in the compiler and runtime—you couldn’t write your own types that contained a `ref`-style reference. This restriction no longer exists. The basic capability that makes spans possible—the ability for a type to have a field that is effectively a `ref` to some other value—is no longer a special power available only to the span types. As [Example 18-13](#) shows, C# 11.0 made it possible to write your own `ref`-like type.

Example 18-13. Type with a `ref` field

```
public readonly ref struct RefLike<T>(ref T rv)
{
    public readonly ref T Ref = ref rv;
}
```

[Example 18-14](#) shows this type in use. The `ri` variable’s `Ref` field is a `ref int` referring to the local variable `i` (because the code passes `ref i` as the constructor argument). When it modifies `ri.Ref` it is really modifying the `i` variable, so the final line displays the value `42`.

Example 18-14. Using a type with a `ref` field

```
int i = 21;
RefLike<int> ri = new(ref i);
ri.Ref *= 2;
Console.WriteLine(i);
```

Only `ref struct` types may contain `ref` fields, for exactly the same reason that only `ref struct` types may contain other `ref struct` types such as spans: types of this kind absolutely must live on the stack, because it would otherwise be possible for a `ref` to some variable in a stack frame to end up in an object or boxed struct on the heap. That would allow the `ref` to survive after the stack frame containing the variable to which it refers no longer exists, which would mean `ref` fields were unsafe. Without this restriction, they'd be no different from pointers, and we've been able to declare pointer-typed fields since C# 1.0. But pointers are inherently unsafe. The whole point of `ref` fields is that they preserve type safety, which is why restrictions on their use exist.

Although the ability to put a `ref` in a field may seem like a relatively simple new feature, the need to ensure type safety meant that the addition of this feature had consequences. In particular, it undermines a simple assumption that used to be true before C# 11.0: if you pass some method a `ref` as an argument, it used to be safe to assume that the method was unable to retain that `ref` after it returned. But the availability of `ref` fields means that when a public method of a `ref struct` type takes an argument of some `ref` type, the caller now has to assume that the `ref struct` might store that `ref` in some `ref` field. This changes the rules about what it is safe to pass in as a `ref` or via a `ref`-like type such as `Span<T>` or the `RefLike<T>` type shown here.

For example, it's normally just fine to pass a reference to a local variable as an argument to a method (either directly as a `ref`, or via a `ref`-like type such as `Span<int>`). This is, in general, safe because the method won't be able to store that reference—it will not be able to use it after it returns. But if the method is a member of a `ref struct`, it *can* stash a reference passed in arguments in a field. The `ChangeTarget` method in [Example 18-15](#) does exactly this. (As it happens, this example doesn't do anything with the stored `ref`, but we could add other members that do.)

Example 18-15. A method that captures a `ref`

```
public ref struct RefSmuggler<T>
{
    private ref T _ref;
```

```

public void ChangeTarget(RefLike<T> rv)
{
    _ref = ref rv.Ref;
}
}

```

Because it's now possible to write code that holds on to a `ref`, the compiler has to apply more conservative rules. [Example 18-16](#) tries to use the `RefSmuggler<T>` type to pass a reference to a local variable back out to its caller. It calls `ChangeTarget` to set the `_ref` field to refer to the `local` variable, but it does this on a `RefSmuggler<T>` passed in by `ref`, which means that `RefSmuggler<T>` will be available to whatever code calls `Bad` after `Bad` returns. This is trying to create a dangling reference—it's trying to give its caller a reference to the `local` variable on its own stack frame, a stack frame that will no longer exist.

Example 18-16. Attempting to enable a `ref` to outlive its target's stack frame

```

static void Bad(ref RefSmuggler<int> rs)
{
    int local = 123;
    RefLike<int> rli = new(ref local);
    rs.ChangeTarget(rli); // Won't compile
}

```

The compiler does not allow this, which is good, but this creates a problem. What if we had a method that takes `ref`-like arguments, but which didn't hold on to any reference, such as [Example 18-17](#)?

Example 18-17. A method that doesn't capture a `ref` but which is handled as if it did

```

public readonly ref struct NoRefCapture<T>
{
    public void UseRef(RefLike<T> rv1, RefLike<T> rv2)
    {
        rv1.Ref = rv2.Ref;
    }
}

```

This doesn't capture a `ref` from either of its arguments. That means that code like [Example 18-18](#) would be safe.

Example 18-18. Code C# thinks might be attempting to enable a `ref` to outlive its target's stack frame

```
void LooksBad(ref NoRefCapture<int> r)
{
    int local1 = 123;
    int local2 = 456;
    RefLike<int> rli1 = new(ref local1);
    RefLike<int> rli2 = new(ref local2);
    r.UseRef(rli1, rli2); // Won't compile
}
```

Unfortunately, the C# compiler won't allow this. It applies exactly the same rules as it did in [Example 18-16](#). We know that in this particular case there isn't really a problem because `UseRef` only makes immediate use of the references passed in, and it doesn't hold on to them after it returns. But the problem is that the C# compiler can't know that in general. If the `NoRefCapture<T>` type is defined in a library, the compiler can't know what's inside the `UseRef` method. (Arguably, it could inspect the IL during compilation, but that would be a bad idea because a future version of the library could change the implementation.) It can only go on the public signature of the method, and from that perspective, there's no real difference between `RefSmuggler<T>.ChangeTarget` and `NoRefCapture<T>.UseRef`. These are both members of `ref struct` types, so either could capture references from their inputs. That's why both examples fail to compile.

This is vexing if the method in question will never have any reason to capture its inputs. It's particularly frustrating if you wrote a library before C# 11.0 came out that includes a method of this kind, because it used to be legal: back when it simply wasn't possible to capture a `ref` there was no problem with this sort of method.

Fortunately, there's a solution. Our method can declare that it will never capture references from specific arguments by marking them with the `scoped` keyword, as [Example 18-19](#) shows. This is part of the method's

public signature, which has two effects. First, the compiler will keep us honest: if we annotated a parameter as `scoped` it will prevent us from attempting to capture a reference obtained through that parameter. Second, code such as [Example 18-18](#) would not cause an error if it used this type. The compiler would know that the references passed in through these arguments won't be captured, so there will be no violation of the type safety rules.

Example 18-19. Declaring non-capture of a `ref`

```
public readonly ref struct NoRefCaptureWithScoped<T>
{
    public void UseRef(scoped RefLike<T> rv1, scoped RefLike<T> rv2)
    {
        rv1.Ref = rv2.Ref;
    }
}
```

Before C# 11.0 it wasn't possible for methods of this form to capture references. It's only the addition of `ref` fields that makes it possible. Code such as [Example 18-18](#) never used to cause compiler errors because it used to be safe. Now it's only safe if the target method declares the argument as `scoped`. But what does that mean if you're using a mixture of old and new code? Your C# 11.0 or 12.0 project might use a library written in C# 10.0 that contains a method similar to [Example 18-19](#). Since the library was written in a version of C# that didn't support `ref` fields, that method can't capture references, but it has parameters that, on current versions of C#, would make it possible to capture references. There was no `scoped` keyword in C# 10.0, so a method such as `UseRef` could only have a signature like the one in [Example 18-17](#). Does that mean that such methods become unusable once you move to C# 11.0 or later?

As you'd probably expect, that's not what happens. If you're using an old library that contains code such as [Example 18-17](#), you'll be able to call it in the way that [Example 18-18](#) does without errors. This works because the compiler now adds attributes to the component to indicate that it was built with a version of the language in which `ref` fields are available. Older libraries (or any library where the project configuration specifies a version of C# older than 11.0) will not have this attribute, so if you're con-

suming such a library from a new project, the compiler will know that the older rules around `ref` handling apply. In effect, all `ref`-like arguments of methods of `ref struct` types defined in older libraries will be handled as though they were `scoped`.

Representing Sequential Elements with `Memory<T>`

The runtime libraries define the `Memory<T>` type and its counterpart, `ReadOnlyMemory<T>`, representing the same basic concept as `Span<T>` and `ReadOnlySpan<T>`. These types provide a uniform view over a contiguous sequence of elements of type `T` that could reside in an array, unmanaged memory, or, if the element type is `char`, a `string`. But unlike spans, these are *not* `ref struct` types, so they can be used anywhere. The downside is that this means they cannot offer the same high performance as spans. (It also means you cannot create a `Memory<T>` that refers to `stackalloc` memory.²)

You can convert a `Memory<T>` to a `Span<T>`, and likewise a `ReadOnlyMemory<T>` to a `ReadOnlySpan<T>`, as long as you're in a context where spans are allowed (e.g., in an ordinary method but not an asynchronous one). The conversion to a span has a cost. It is not massive, but it is significantly higher than the cost of accessing an individual element in a span. (In particular, many of the optimizations that make spans attractive only become effective with repeated use of the same span.)

So if you are going to read or write elements in a `Memory<T>` in a loop, you should perform the conversion to `Span<T>` just once, outside of the loop, rather than doing it each time around. If you can work entirely with spans, you should do so since they offer the best performance. (And if you are not concerned with performance, then this is not the chapter for you!)

ReadOnlySequence<T>

The types we've looked at so far in this chapter all represent contiguous blocks of memory. Unfortunately, data doesn't always neatly present itself to us in the most convenient possible form. For example, on a busy server that is handling many concurrent requests, the network messages for requests in progress often become interleaved—if a particular request is large enough to need to be split across two network packets, it's entirely possible that after receiving the first but before receiving the second of these, one or more packets for other, unrelated requests could arrive. So by the time we come to process the contents of the request, it might be split across two different chunks of memory. Since span and memory values can each represent only a contiguous range of elements, .NET provides another type, `ReadOnlySequence`, to represent data that is conceptually a single sequence but that has been split into multiple ranges.

NOTE

There is no corresponding `Sequence<T>`. Unlike spans and memory, this particular abstraction is available only in read-only form. That's because it's common to need to deal with fragmented data as a reader, where you don't control where the data lives, but if you are producing data, you are more likely to be in a position to control where it goes.

Now that we've seen the main types for working with data while minimizing the number of allocations, let's look at how these can all work together to handle high volumes of data. To coordinate this kind of processing, we need to look at one more feature: pipelines.

Processing Data Streams with Pipelines

Everything we're looking at in this chapter is designed to enable safe, efficient processing of large volumes of data. The types we've seen so far all represent information that is already in memory. We also need to think about how that data is going to get into memory in the first place. The preceding section hinted at the fact that this can be somewhat messy. The

data will very often be split into chunks, and not in a way designed for the convenience of the code processing the data, because it will likely be arriving either over a network or from a disk. If we're to realize the performance benefits made possible by `Span<T>` and its related types, we need to pay close attention to the job of getting data into memory in the first place and the way in which this data fetching process cooperates with the code that processes the data. Even if you are only going to be writing code that consumes data—perhaps you are relying on a framework such as ASP.NET Core to get the data into memory for you—it is important to understand how this process works.

The `System.IO.Pipelines` NuGet package defines a set of types in a namespace of the same name that provide a high-performance system for loading data from some source that tends to split data into inconveniently sized chunks, and passing that data over to code that wants to be able to process it in situ using spans. [Figure 18-2](#) shows the main participants in a pipeline-based process.

At the heart of this is the `Pipe` class. It offers two properties: `Writer` and `Reader`. The first returns a `PipeWriter`, which is used by the code that loads the data into memory. (This often doesn't need to be application-specific. For example, in a web application, you can let ASP.NET Core control the writer on your behalf.) The `Reader` property's type is, predictably, `PipeReader`, and this is most likely to be the part your code interacts with.

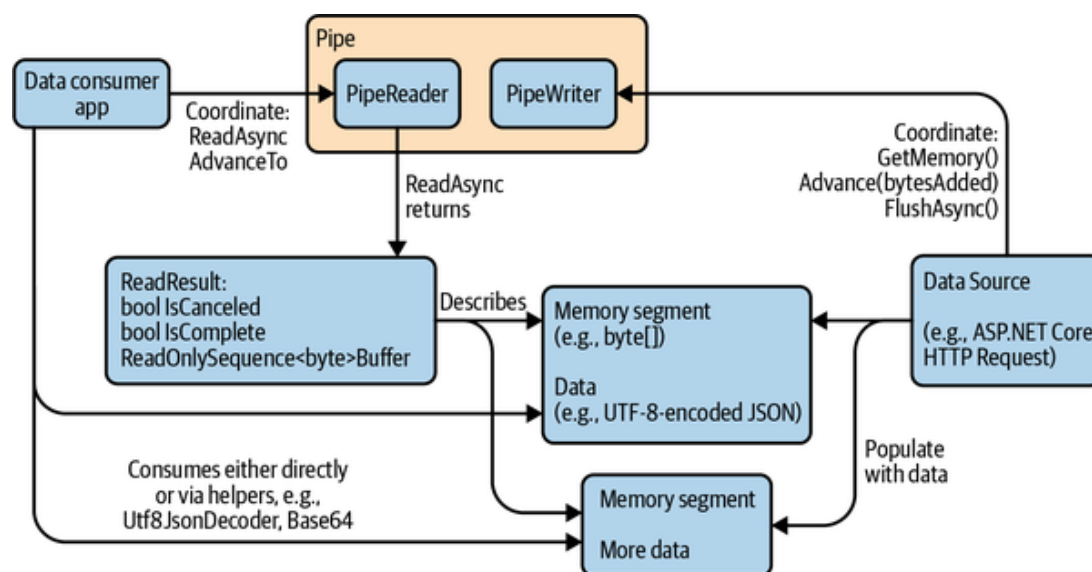


Figure 18-2. Pipeline overview

The basic process for reading data from a pipe is as follows. First, you call `PipeReader.ReadAsync`. This returns a task,³ because if no data is available yet, you will need to wait until the data source supplies the writer with some data. Once data is available, the task will provide a `ReadResult` object. This supplies a `ReadOnlySequence<T>`, which presents the available data as one or more `ReadOnlySpan<T>` values. The number of spans will depend on how fragmented the data is. If it's all conveniently in one place in memory, there will be just one span, but code using a reader needs to be able to cope with more. Your code should then process as much of the available data as it can. Once it has done this, it calls the reader's `AdvanceTo` to tell it how much of the data your code has been able to process. Then, if the `ReadResult.IsComplete` property is false, we will repeat these steps again from the call to `ReadAsync`.

An important detail of this is that we are allowed to tell the `PipeReader` that we couldn't process everything it gave us. This would normally be because the information got sliced into pieces, and we need to see some of the next chunk before we can fully process everything in the current one. For example, a JSON message large enough to need to be split across several network packets will probably end up with splits in inconvenient places. So you might find that the first chunk looks like this:

```
{"property1":"value1", "prope
```

And the second like this:

```
rty2":42}
```

In practice the chunks would be bigger, but this illustrates the basic problem: the chunks that a `PipeReader` returns are likely to slice across the middle of important features. With most .NET APIs, you never have to deal with this kind of mess because everything has been cleaned up and reassembled by the time you see it, but the price you pay for that is the allocation of new strings to hold the recombined results. If you want to avoid those allocations, you have to handle these challenges.

There are a couple of ways to deal with this. One is for code reading data to maintain enough state to be able to stop and later restart at any point

in the sequence. So code processing this JSON might choose to remember that it is partway through an object and that it's in the middle of processing a property whose name starts with `prope`. But `PipeReader` offers an alternative. Code processing these examples could report with its call to `AdvanceTo` that it has consumed everything up to the first comma. If you do that, the `Pipe` will remember that we're not yet finished with this first block, and when the next call to `ReadAsync` completes, the `ReadOnlySequence<T>` in `ReadResult.Buffer` will now include at least two spans: the first span will point into the same block of memory as last time, but now its offset will be set to where we got to last time—that first span will refer to the `"prope` text at the end of the first block. And then the second span will refer to the text in the second chunk.

The advantage of this second approach is that the code processing the data doesn't need to remember as much between calls to `ReadAsync`, because it knows it'll be able to go back and look at the previously unprocessed data again once the next chunk arrives, at which point it should now be able to make sense of it.

In practice, this particular example is fairly easy to cope with because there's a type in the runtime libraries called `Utf8JsonReader` that can handle all the awkward details around chunk boundaries for us. Let's look at an example.

Processing JSON in ASP.NET Core

Suppose you are developing a web service that needs to handle HTTP requests containing JSON. This is a pretty common scenario. [Example 18-20](#) shows a common way to do this in ASP.NET Core. This is reasonably straightforward, but it does not use any of the low-allocation mechanisms discussed in this chapter, so this forces ASP.NET Core to allocate multiple objects for each request.

Example 18-20. Handling JSON in HTTP requests

```
[HttpPost]
[Route("/jobs/create")]
public void CreateJob([FromBody] JobDescription requestBody)
```

```

{
    switch (requestBody.JobCategory)
    {
        case "arduous":
            CreateArduousJob(requestBody.DepartmentId);
            break;

        case "tedious":
            CreateTediousJob(requestBody.DepartmentId);
            break;
    }
}

public record JobDescription(int DepartmentId, string JobCategory);

```

Before we look at how to change it, for readers not familiar with ASP.NET Core, I will quickly explain what's happening in this example. The `CreateJob` method is annotated with attributes telling ASP.NET Core that this will handle HTTP POST requests where the URL path is `/jobs/create`. The `[FromBody]` attribute on the method's argument indicates that we expect the body of the request to contain data in the form described by the `JobDescription` type. ASP.NET Core can be configured to handle various data formats, but if you go with the defaults, it will expect JSON.

This example is therefore telling ASP.NET Core that for each POST request to `/jobs/create`, it should construct a `JobDescription` object, populating its `DepartmentId` and `JobCategory` from properties of the same names in JSON in the incoming request body.

In other words, we're asking ASP.NET Core to allocate two objects—a `JobDescription` and a `string`—for each request, each of which will contain copies of information that was in the body of the incoming request. (The other property, `DepartmentId`, is an `int`, and since that's a value type, it lives inside the `JobDescription` object.) And for most applications that will be fine—a couple of allocations is not normally anything to worry about in the course of handling a single web request. However, in more realistic examples with more complex requests, we might then be looking at a much larger number of properties, and if you need to handle a very high volume of requests, the copying of data into a `string` for each prop-

erty can start to cause enough extra work for the GC that it becomes a performance problem.

[Example 18-21](#) shows how we can avoid these allocations using the various features described in the preceding sections of this chapter. It makes the code a good deal more complex, demonstrating why you should only apply these kinds of techniques in cases where you have established that GC overhead is high enough that the extra development effort is justified by the performance improvements.

Example 18-21. Handling JSON without allocations

```
[HttpPost]
[Route("/jobs/create")]
public async ValueTask CreateJobFrugalAsync()
{
    bool inDepartmentIdProperty = false;
    bool inJobCategoryProperty = false;
    int? departmentId = null;
    bool? isArduous = null;

    PipeReader reader = this.Request.BodyReader;
    JsonReaderState jsonState = default;
    while (true)
    {
        ReadResult result = await reader.ReadAsync().ConfigureAwait(false);
        jsonState = ProcessBuffer(
            result,
            jsonState,
            out SequencePosition position);

        if (departmentId.HasValue && isArduous.HasValue)
        {
            if (isArduous.Value)
            {
                CreateArduousJob(departmentId.Value);
            }
            else
            {
                CreateTediousJob(departmentId.Value);
            }
        }

        return;
    }
}
```

```

    }

    reader.AdvanceTo(position);

    if (result.IsCompleted)
    {
        break;
    }
}

JsonReaderState ProcessBuffer(
    in ReadResult result,
    in JsonReaderState jsonState,
    out SequencePosition position)
{
    // This is a ref struct, so this has no GC overhead
    var r = new Utf8JsonReader(result.Buffer, result.IsCompleted, jsonState);

    while (r.Read())
    {
        if (inDepartmentIdProperty)
        {
            if (r.TokenType == JsonTokenType.Number)
            {
                if (r.TryGetInt32(out int v))
                {
                    departmentId = v;
                }
            }
        }
        else if (inJobCategoryProperty)
        {
            if (r.TokenType == JsonTokenType.String)
            {
                if (r.ValueSpan.SequenceEqual("arduous"u8))
                {
                    isArduous = true;
                }
                else if (r.ValueSpan.SequenceEqual("tedious"u8))
                {
                    isArduous = false;
                }
            }
        }
    }
}

```

```

        inDepartmentIdProperty = false;
        inJobCategoryProperty = false;

        if (r.TokenType == JsonTokenType.PropertyName)
        {
            if (r.ValueSpan.SequenceEqual("JobCategory"u8))
            {
                inJobCategoryProperty = true;
            }
            else if (r.ValueSpan.SequenceEqual("DepartmentId"u8))
            {
                inDepartmentIdProperty = true;
            }
        }
    }

    position = r.Position;
    return r.CurrentState;
}
}

```

Instead of defining an argument with a `[FromBody]` attribute, this method works directly with the `this.Request.BodyReader` property. (Inside an ASP.NET Core MVC controller class, `this.Request` returns an object representing the request being handled.) This property's type is `PipeReader`, the consumer side of a `Pipe`. ASP.NET Core creates the pipe, and it manages the data production side, feeding data from incoming requests into the associated `PipeWriter`.

As the property name suggests, this particular `PipeReader` enables us to read the contents of the HTTP request's body. By reading the data this way, we make it possible for ASP.NET Core to present the request body to us in situ: our code will be able to read the data directly from wherever it happened to end up in memory once the computer's network card received it. (In other words, no copies, and no additional GC overhead.)

The `while` loop in `CreateJobFrugalAsync` performs a process common to any code that reads data from a `PipeReader`: it calls `ReadAsync`, processes the data that returns, and calls `AdvanceTo` to let the `PipeReader` know how much of that data it was able to process. We then check the

`IsComplete` property of the `ReadResult` returned by `ReadAsync`, and if that is `false`, then we go round one more time.

[Example 18-21](#) uses the `Utf8JsonReader` type to process the data. As the name suggests, this works directly with text in UTF-8 encoding. JSON messages are commonly sent with this encoding, but .NET strings use UTF-16. So one of the jobs that the simpler [Example 18-20](#) forced ASP.NET to do was convert any strings from UTF-8 to UTF-16. Avoiding this conversion can provide a significant performance improvement, although it does lose some flexibility. The simpler, slower approach has the benefit of being able to adapt to incoming requests in more formats: if a client chose to send its request in something other than UTF-8—perhaps UTF-16 or UCS-32, or even a non-Unicode encoding such as ISO-8859-1—our first handler could cope with any of them, because ASP.NET Core can do the string conversions for us. But since [Example 18-21](#) works directly with the data in the form the client transmitted, using a type that only understands UTF-8, we have traded off that flexibility in exchange for higher performance.

`Utf8JsonReader` is able to handle the tricky chunking issues for us—if an incoming request ends up being split across multiple buffers in memory because it was too large to fit in a single network packet,

`Utf8JsonReader` is able to cope. In the event of an unhelpfully placed split, it will process what it can, and then the `JsonReaderState` value it returns through its `CurrentState` will report a `Position` indicating the first unprocessed character. We pass this to `PipeReader.AdvanceTo`. The next call to `PipeReader.ReadAsync` will return only when there is more data, but its `ReadResult.Buffer` will also include the previously unconsumed data.

Like the `ReadOnlySpan<T>` type it uses internally when reading data, `Utf8JsonReader` is a `ref struct` type, meaning that it cannot live on the heap. This means it cannot be used in an `async` method, because `async` methods store all of their local variables on the heap. That is why this example has a separate method, `ProcessBuffer`. The outer `CreateJobFrugalAsync` method has to be `async` because the streaming nature of the `PipeReader` type means that its `ReadAsync` method requires us to use `await`. But the `Utf8JsonReader` cannot be used in an

`async` method, so we end up having to split our logic across two methods.

When splitting your pipeline processing into an outer `async` reader loop and an inner method that avoids `async` in order to use `ref struct` types, it can be convenient to make the inner method a local method, as [Example 18-21](#) does. This enables it to access variables declared in the outer method. You might be wondering whether this causes a hidden extra allocation—to enable sharing of variables in this way, the compiler generates a type, storing shared variables in fields in that type and not as conventional stack-based variables. With lambdas and other anonymous methods, this type will indeed cause an additional allocation, because it needs to be a heap-based type so that it can outlive the parent method. However, with local methods, the compiler uses a `struct` to hold the shared variables, which it passes by reference to the inner method, thus avoiding any extra allocation. This is possible because the compiler can determine that all calls to the local method will return before the outer method returns.

When using `Utf8JsonReader`, our code has to be prepared to receive the content in whatever order it happens to arrive. We can't write code that tries to read the properties in an order that is convenient for us, because that would rely on something holding those properties and their values in memory. (If you tried to rely on going back to the underlying data to retrieve particular properties on demand, you might find that the property you wanted was in an earlier chunk that's no longer available.) This defeats the whole goal of minimizing allocations. If you want to avoid allocations, your code needs to be flexible enough to handle the properties in whatever order they appear.

So the `ProcessBuffer` code in [Example 18-21](#) just looks at each JSON element as it comes and works out whether it's of interest. This means that when looking for particular property values, we have to notice the `PropertyName` element, and then remember that this was the last thing we saw, so that we know how to handle the `Number` or `String` element that follows, containing the value.

One strikingly odd feature of this code is the way it checks for particular strings. It needs to recognize properties of interest (`JobCategory` and

`DepartmentId` in this example) but it doesn't just use normal string comparison. While it's possible to retrieve property names and string values as .NET strings, doing so defeats the main purpose of using `Utf8JsonReader`: if you obtain a `string`, the CLR has to allocate space for that string on the heap and will eventually have to garbage collect the memory. (In this example, every acceptable incoming string is known in advance. In some scenarios there will be strings in the incoming data whose values you will need to perform further processing on, and in those cases, you may just need to accept the costs of allocating an actual `string`.) So instead we end up performing binary comparisons by calling `r.ValueSpan.SequenceEqual`. Notice that we're working entirely in UTF-8 encoding, and not the UTF-16 encoding used by .NET's `string` type. (The various strings passed to `SequenceEqual` use the UTF8 string literal syntax introduced in C# 11.0—they all have a `u8` suffix. As you saw in [“UTF-8 string literals”](#) this means that instead of producing `string` objects, the compiler embeds the UTF-8 representation of these strings directly into the compiled component, and creates `ReadOnlySpan<byte>` values pointing to the data.) That's because all of this code works directly against the request's payload in the form in which it arrived over the network, in order to avoid unnecessary copying.

Summary

APIs that break data down into the constituent components can be very convenient to use, but this convenience comes at a price. Each time we want some subelement represented either as a string or a child object, we cause another object to be allocated on the GC heap. The cumulative cost of these allocations (and the corresponding work to recover the memory once they are no longer in use) can be damaging in some very performance-sensitive applications. They can also be significant in cloud applications or high-volume data processing, where you might be paying for the amount of processing work you do—reducing CPU or memory usage can have a nontrivial effect on cost.

The `Span<T>` type and the related types discussed in this chapter make it possible to work with data wherever it already resides in memory. This typically requires rather more complex code, but in cases where the pay-

off justifies the work, these features make it possible for C# to tackle whole classes of problems for which it would previously have been too slow.

Thank you for reading this book, and congratulations for making it to the end. I hope you enjoy using C#, and I wish you every success with your future projects.

- ¹ That said, it is possible to perform this kind of conversion explicitly—the `MemoryMarshal` class offers methods that can take a span of one type and return another span that provides a view over the same underlying memory, interpreted as containing a different element type. But it is unlikely to be useful in this case: converting a `ReadOnlySpan<char>` to a `ReadOnlySpan<int>` would produce a span with half the number of elements, where each `int` contained pairs of adjacent `char` values.
- ² Technically you could write a custom `MemoryManager` to do this, but the compiler and runtime would be unable to enforce safety if you did that.
- ³ It is a `ValueTask<ReadResult>` because the purpose of this exercise is to minimize allocations. `ValueTask<T>` was described in [Chapter 16](#).